

Machine Learning II: Explainable Artificial Intelligence (XAI)

Week 8, Day 2: Predicting Atherosclerotic Heart Disease

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

6th Oct 2026

Abstract

This presentation explores the application of Ensemble Machine Learning models and Explainable Artificial Intelligence (XAI) to predict Atherosclerotic Heart Disease (AHD).

- **Clinical Objective:** Develop robust predictive models to enable early intervention and risk stratification for cardiac patients.
- **Modeling Strategy:** Compare Random Forest and XGBoost classifiers using rigorous hyperparameter tuning and stratified cross-validation.
- **Evaluation Framework:** Assess models using discrimination (ROC-AUC), class imbalance sensitivity (PR-AUC), and probability calibration.
- **Explainability (XAI):** Utilize SHAP (SHapley Additive exPlanations) to demystify "black-box" predictions, translating complex algorithmic outputs into actionable clinical insights.

Session Objectives

- Formulate a clinical machine learning problem and define the cost of false positives vs. false negatives.
- Construct robust, leakage-free preprocessing pipelines for mixed data types (numerical and categorical).
- Implement and tune ensemble models (Random Forest and XGBoost) handling class imbalance.
- Evaluate model performance using advanced metrics beyond simple accuracy (PR-AUC, Calibration).
- Apply SHAP waterfall plots to explain individual patient predictions and build trust with clinical stakeholders.

Part 1: Problem Formulation

"Defining the clinical objective and costs."

Machine Learning & XAI Overview

- **Machine Learning (ML):** Enables computers to learn patterns from data without explicit programming. Branches include Supervised, Unsupervised, Semi-Supervised, and Reinforcement Learning.
- **The "Black Box" Problem:** Complex models like XGBoost offer high accuracy but lack transparency.
- **Explainable AI (XAI):** A suite of methods (e.g., SHAP, LIME) designed to make ML predictions transparent, interpretable, and trustworthy.
- **Why XAI in Healthcare?** Clinicians must understand *why* a model flags a patient as high-risk before acting. It fosters trust, enables error detection, and supports shared decision-making.

Problem Formulation: AHD Prediction

Clinical Objective

Predict the presence of Atherosclerotic Heart Disease (AHD) to enable early intervention and risk stratification.

- **Target Variable:** `AHD` (Binary: 0 = No Disease, 1 = Disease Present).
- **Hypotheses:**
  - XGBoost will outperform RF in discrimination due to gradient optimization.
  - Key predictors will include structural test results (Thal, Ca) over basic vitals.
  - Dynamic class weighting will mitigate bias toward the majority class.

Real-World Clinical Implications

The Cost of Errors

- **False Negatives (Missed Risk):** Delayed treatment, potential acute events (MI, stroke). **HIGH clinical cost.** $\rightarrow$ *Recall/Sensitivity is prioritized.*
- **False Positives (Over-diagnosis):** Unnecessary angiography/stress tests, patient anxiety, healthcare costs. **MODERATE clinical cost.** $\rightarrow$ *Precision balances this.*

- **Calibration:** Clinicians require well-calibrated probabilities (not just raw scores) to make shared decisions with patients regarding invasive procedures.

Part 2: Data Import & Exploration

"Understanding the clinical markers."

Loading Libraries & Dataset

Python Implementation

```
import pandas as pd, numpy as np, shap, xgboost as xgb
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, RandomizedSearchCV
# ... (other imports for metrics and pipelines)

# Load data with explicit NA handling
df = pd.read_csv('Heart.csv', na_values=['NA', '?', ''])
df['AHD'] = df['AHD'].map({'No': 0, 'Yes': 1}) # Encode target

```

Code Explanation

- We explicitly define missing value indicators (‘NA‘, ‘¿, ‘"""‘) during import to ensure pandas correctly flags them as ‘NaN‘.
- The target variable ‘AHD‘ is mapped to binary integers (0 and 1) for compatibility with scikit-learn classifiers.

Dataset Overview & Key Variables

Variable	Definition & Clinical Relevance	Type
Age	Patient's age; strong predictor of CVD risk.	Cont.
ChestPain	Type of pain; asymptomatic indicates severe disease.	Cat.
MaxHR	Max heart rate; lower values indicate poor function.	Cont.
Oldpeak	ST depression; marker of myocardial ischemia.	Cont.
Ca	Major vessels colored; more vessels = severe disease.	Ord.
Thal	Thalassemia test; fixed/reversible defects = ischemia.	Cat.
AHD	Target: Atherosclerotic Heart Disease (0=No, 1=Yes).	Bin.

EDA: Target Distribution & Missing Values

Python Implementation

```
# Target Distribution
print(df['AHD'].value_counts())
# Output: 0 (No): 164, 1 (Yes): 139

# Missing Values
print(df.isnull().sum()[df.isnull().sum() > 0])
# Output: Ca: 4, Thal: 2

```

Graph & Data Explanation

- **Target Balance:** The dataset is relatively balanced (54% No Disease, 46% Disease), which is ideal for training without extreme oversampling.
- **Missingness:** Only 'Ca' (4 missing) and 'Thal' (2 missing) have gaps. These are critical structural markers, so we will impute them carefully in the pipeline rather than dropping rows.

EDA: Numerical & Categorical Features

Graph Explanations (Boxplots & Stacked Bar Charts)

- **Numerical Boxplots:** Reveal that ‘MaxHR‘ is significantly lower in the HD=1 group, while ‘Oldpeak‘ and ‘Ca‘ are shifted higher. This confirms their strong discriminatory power.
- **Categorical Stacked Bars:** Show the incidence rate of HD within categories.
  - `ChestPain` type "asymptomatic" shows a massive spike in HD probability.
  - `Thal` "reversible defect" heavily correlates with positive AHD cases.

Insight

Structural and functional test results (‘Thal‘, ‘Ca‘, ‘ChestPain‘) appear to be stronger drivers of disease presence than general vitals like resting blood pressure.

EDA: Correlation & Multivariate Analysis

Graph Explanations (Heatmap & Pairplot)

- **Correlation Heatmap:** Identifies multicollinearity. We mask the upper triangle for clarity. High correlations between 'Age' and 'MaxHR' (negative) are expected physiologically.
- **Multivariate Pairplot:** Visualizes interactions in 2D space.
  - **Ca & Thal Interaction:** Patients with $Ca > 0$ AND a specific 'Thal' defect almost exclusively fall into the 'HD=1' category.
  - **Age vs MaxHR:** For a given age, HD patients tend to have an abnormally low MaxHR compared to healthy patients.

Part 3: Preprocessing & Partitioning

"Building robust, leakage-free pipelines."

Data Preprocessing Pipeline

Python Implementation

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(handle_unknown='use_encoded_value',
                               unknown_value=-1))])

preprocessor = ColumnTransformer(
    transformers=[('num', numeric_transformer, num_features),
                  ('cat', categorical_transformer, cat_features)],
    remainder='drop')

```

Code Explanation

We use ‘ColumnTransformer‘ to apply distinct imputation and scaling logic to numerical vs. categorical features, ensuring tree models and distance-based metrics receive clean, encoded data.

Data Partitioning Strategy

Python Implementation

```
X = df.drop(columns=['AHD', 'HD']) # Drop redundant target
y = df['AHD']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

```

Code Explanation

- **Stratification:** 'stratify=y' ensures the 54/46 class split is preserved in both train and test sets.
- **Leakage Prevention:** The 'preprocessor' is **not** fitted yet. It will only be fitted inside the cross-validation loops and final model training to prevent test data statistics from leaking into the training process.

Verifying Class Proportionality

Class (AHD)	Train (%)	Test (%)
0 (No Disease)	54.1%	54.1%
1 (Disease)	45.9%	45.9%

Table: Class Distribution Across Partitions

Inferential Test (Chi-Square)

- **Chi-square Statistic:** 0.0000
- **P-value:** 1.0000
- **Conclusion:** The p-value > 0.05 confirms there is no statistically significant difference in class proportions between the train and test sets. The stratified split was perfectly executed.

Part 4: Model Training & Tuning

"Optimizing Random Forest and XGBoost."

Handling Class Imbalance & Pipeline Setup

Python Implementation

```
# Calculate imbalance ratio for XGBoost
pos_weight_train = (y_train == 0).sum() / (y_train == 1).sum()

rf_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('rf', RandomForestClassifier(random_state=42, n_jobs=-1))])

xgb_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('xgb', xgb.XGBClassifier(random_state=42, eval_metric='logloss',
                              scale_pos_weight=pos_weight_train))])

```

Code Explanation

- `scale_pos_weight` in XGBoost penalizes misclassifications of the minority class more heavily, forcing the model to pay closer attention to detecting AHD.
- Random Forest will handle imbalance via the 'class_weight' hyperparameter during tuning.

Hyperparameter Tuning: Random Forest

Search Space Definition

- `n_estimators`: [100, 200, 300, 400] – Number of trees.
- `max_depth`: [None, 10, 15, 20] – Limits tree complexity to prevent overfitting.
- `min_samples_split` & `min_samples_leaf`: Controls node splitting conservatism.
- `class_weight`: ['balanced', 'balanced_subsample'] – Crucial for adjusting to the 54/46 split dynamically per tree.

Strategy

Random Forest parameters mainly control tree complexity and splitting behavior. We use 'RandomizedSearchCV' to sample 30 combinations efficiently rather than an exhaustive grid search.

Hyperparameter Tuning: XGBoost

Search Space Definition

- `n_estimators`: [100, 200, 300] – Boosting rounds.
- `max_depth`: [3, 4, 5, 6] – Kept shallow to avoid overfitting.
- `learning_rate`: [0.01, 0.05, 0.1, 0.2] – Step size shrinkage.
- `subsample` & `colsample_bytree`: Introduces randomness (stochastic gradient boosting).
- `gamma` & `reg_alpha`: L1 regularization and minimum loss reduction to enforce sparsity and conservatism.

Strategy

XGBoost parameters control sequential error correction, regularization, and randomness.

Model Training Process

Python Implementation

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

search_rf = RandomizedSearchCV(estimator=rf_pipeline,
    param_distributions=param_dist_rf, n_iter=30, cv=cv,
    scoring='roc_auc', n_jobs=-1, random_state=42)
search_rf.fit(X_train, y_train)

```

Code Explanation

- **Stratified 5-Fold CV:** Ensures every fold maintains the target ratio.
- **Scoring Metric:** We optimize for `roc_auc` because accuracy is misleading in medical diagnostics where discrimination across all thresholds matters most.
- **Pipeline Integrity:** Because 'search, rf 'wrapsthepipeline, the'preprocessor'issafelyrefit5timesperiteration, completelyeliminatingdataleakage.

Part 5: Prediction & Evaluation

"Assessing clinical utility and discrimination."

Hold-Out Test Set Evaluation

Python Implementation

```
# Generate predictions and probabilities
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Calculate Precision-Recall AUC (crucial for clinical tasks)
prec, rec, _ = precision_recall_curve(y_test, y_prob)
pr_auc = auc(rec, prec)

```

Code Explanation

- We evaluate on the untouched 20% hold-out set.
- **PR-AUC (Average Precision):** While ROC-AUC evaluates overall ranking, PR-AUC focuses heavily on the model's ability to correctly identify the positive class (Disease) without drowning in false positives. This is the gold standard for imbalanced medical tasks.

Performance Metrics Comparison

Metric	Random Forest	XGBoost
Accuracy	0.9016	0.9016
Precision	0.8667	0.8667
Recall	0.9286	0.9286
F1-Score	0.8966	0.8966
ROC-AUC	**0.9643**	0.9578
PR-AUC	**0.9539**	0.9427

Table: Hold-Out Test Set Evaluation Metrics

Key Takeaway

Both models exceed 90% across almost all metrics. Random Forest shows slightly superior discrimination (ROC-AUC) and Precision-Recall balance.

Discriminative Power: ROC & PR Curves

Graph Explanations

- **ROC Curves (AUC $\approx$ 0.96):** Both models hug the top-left corner tightly. This indicates excellent separability; they maintain a high True Positive Rate while keeping the False Positive Rate exceptionally low.
- **Precision-Recall Curves (AP $\approx$ 0.95):** Random Forest performs slightly better here. The RF curve stays closer to the top-right corner (Precision=1, Recall=1) for longer than XGBoost, proving it is slightly more reliable when the cost of false alarms is considered.

Reliability: Calibration & Learning Curves

Graph Explanations

- **Calibration Curves (Reliability Diagrams):**
  - **XGBoost:** Jumps rapidly to 1.0. Very confident, but slightly miscalibrated in the mid-range.
  - **Random Forest:** Follows a smoother trajectory, tracking closer to the perfect diagonal. It is more "conservative" and reliable for risk stratification.
- **Learning Curves (Bias-Variance):** Training score remains high ($\sim$0.98), while validation score plateaus around 0.89. The visible gap indicates a slight tendency toward overfitting, but generalization remains strong enough for clinical deployment.

Part 6: Feature Importance

"What drives the clinical predictions?"

Feature Importance Analysis (XGBoost Gain)

Feature	Importance	Feature	Importance
Thal	0.300	ExAng	0.076
Ca	0.147	Sex	0.058
ChestPain	0.144	Slope	0.058
Oldpeak	0.049	Age	0.034
MaxHR	0.038	RestBP	0.026

Graph Explanation (Bar Chart)

The horizontal bar chart reveals that structural and functional test features (‘Thal‘, ‘Ca‘, ‘ChestPain‘) dominate predictive importance, accounting for nearly 60% of the model’s decision-making power.

Clinical Insights from Features

- **Thal (Thalassemia Stress Test):** The most significant predictor ($\sim$30% gain). A fixed or reversible defect is a massive indicator of ischemia.
- **Ca (Major Vessels Colored):** Direct anatomical indicator of blockage severity. $Ca \geq 2$ almost guarantees a high-risk flag.
- **ChestPain:** "Asymptomatic" pain is counter-intuitively dangerous; it often indicates severe, silent ischemia.
- **Vitals vs. Tests:** Standard metrics like ‘Chol‘ and ‘RestBP‘ have surprisingly low importance. **Insight:** Targeted cardiac diagnostics outweigh general baseline vitals in predicting acute AHD.

Selection of the Best Model

Final Recommendation: Random Forest Pipeline

- **Quantitative Superiority:** Achieved higher ROC-AUC (0.9643) and PR-AUC (0.9539).
- **Clinical Calibration:** Smoother calibration curve implies more reliable probability estimates for shared decision-making.
- **Safety:** High recall (0.9286) ensures very few cardiac events are missed.

Deployment Strategy

Deploy with a recall-optimized decision threshold. Predicted probabilities must be presented alongside SHAP explanations to support clinician trust.

Part 7: Explainable AI (SHAP)

"Opening the black box for individual patients."

Introduction to Explainable AI (XAI)

- **The Problem:** A clinician will not order an invasive angiogram based solely on a "black box" outputting ‘0.91‘. They need to know *why*.
- **SHAP (SHapley Additive exPlanations):** Based on cooperative game theory. It assigns each feature an importance value for a *particular* prediction.
- **Waterfall Plots:** Visualize how the model starts at a baseline population risk and adds/subtracts evidence (features) to arrive at the final patient-specific probability.

SHAP Extraction for Random Forest

Python Implementation

```
# Extract model and preprocessor from pipeline
rf_model = full_pipeline.named_steps['rf']
preprocessor = full_pipeline.named_steps['preprocessor']
X_test_trans = preprocessor.transform(X_test)

# Initialize TreeExplainer and compute values
explainer_rf = shap.TreeExplainer(rf_model)
shap_values_rf = explainer_rf.shap_values(X_test_trans)

# Safely extract positive class (Class 1: Disease)
shap_values_pos = shap_values_rf[1]
base_val = explainer_rf.expected_value[1]

```

Interpreting the SHAP Waterfall Plot (RF)

Graph Explanation (Waterfall Plot)

- $E[f(x)] = 0.503$ **(Base Value):** The average predicted probability of AHD across the training set. The model starts with a 50.3% baseline assumption.
- **Red Bars (Pushing Right):** Features increasing risk. E.g., 'Ca = 3' might push probability up by $+0.35$.
- **Blue Bars (Pushing Left):** Features decreasing risk. E.g., 'MaxHR = 180' might pull probability down by -0.05.
- $f(x) = 0.913$ **(Final Output):** The sum of the base value and all SHAP pushes equals the final 91.3% risk prediction for this specific patient.

Clinical Translation: The Courtroom Analogy

- **The Baseline ($E[f(x)]$):** The starting assumption based on general hospital statistics ("About half the patients we test have heart disease").
- **The Evidence (Red/Blue Bars):** The specific pieces of evidence presented for *this* patient.
  - "The fluoroscopy shows 3 blocked vessels (strong evidence for disease)."
  - "However, their cholesterol is normal (evidence against)."
- **The Verdict ($f(x)$):** The final probability after weighing all evidence. "Based on the heavy evidence of blocked vessels, we are 91.3% certain this patient has the disease."

SHAP Extraction for XGBoost

Python Implementation

```
xgb_model = full_pipeline.named_steps['xgb']
explainer_xgb = shap.TreeExplainer(xgb_model)
shap_values_xgb = explainer_xgb.shap_values(X_test_trans)

# XGBoost natively outputs log-odds in SHAP
base_val_xgb = explainer_xgb.expected_value

```

Key Difference

Unlike Random Forest, which averages direct probabilities, XGBoost optimizes using "log-loss". Therefore, its raw mathematical output (and SHAP values) are inherently in **log-odds**, not direct percentages.

XGBoost SHAP: Log-Odds vs Probabilities

- **Random Forest** outputs direct probabilities (0.0 to 1.0).
- **XGBoost** outputs raw **log-odds** ($-\infty$ to $+\infty$).
- A log-odds of 0 equals exactly a 50% probability.
- **Baseline Risk ($E[f(x)]$):** For XGBoost, the baseline might appear as -0.002.
- Converting to probability via the sigmoid function:

$$\text{Probability} = \frac{1}{1 + e^{-\text{log-odds}}} = \frac{1}{1 + e^{-(-0.002)}} \approx 0.4995 \tag{1}$$

Clinical Meaning

A baseline probability of $\sim 50\%$ perfectly aligns with our relatively balanced dataset.

Mathematical Verification of XGBoost

Verifying the Waterfall Plot

- Look at the final value at the bottom of the XGBoost waterfall plot, labeled $f(x)$ (e.g., $f(x) = 2.5$).
- Plug that log-odds value into the sigmoid formula:

$$\text{Probability} = \frac{1}{1 + e^{-2.5}} \approx 0.924 \text{ (or } 92.4\%) \tag{2}$$

Teaching Tip

This mathematical verification proves that SHAP isn't guessing; it is perfectly deconstructing the exact raw, native output of the XGBoost algorithm. The red and blue bars are adding and subtracting *log-odds*, not percentages.

Final Conclusion & Policy Recommendations

- **Summary:** Ensemble models achieve exceptional diagnostic performance (ROC-AUC > 0.95). Structural features ('Thal', 'Ca') dominate predictive importance over basic vitals.
- **Clinical Deployment:** Deploy Random Forest with SHAP explanations to prioritize sensitivity and support shared decision-making.
- **Policy Recommendations:**
  - Invest in standardized data capture for high-value features (e.g., fluoroscopy vessel counts).
  - Mandate prospective validation on diverse cohorts before deployment.
  - Integrate interpretable AI literacy into clinical training programs.

Victor Wandera Lumumba
Statistician | Data Analyst | ML Expert
Mediacrest Training College